

Bug Hunting & Transaction Integrity Audit

Sistem Kasir Toko — audit adversarial atas integritas transaksi

Proyek	kasir-toko (Next.js 15 · TypeScript strict · Prisma/SQLite)
Tanggal audit	21 September 2026
Versi laporan	1.0
Commit dasar	ad95e94
Ruang lingkup	Checkout, refund, pengeluaran, barang masuk, penyesuaian stok, jasa pembayaran, saldo provider, QRIS, shift,
Server presentasi	RUNNING — PID 16412, port 3000, tidak pernah dimatikan
Metode	Seluruh serangan dijalankan pada server dan database TERPISAH (port bebas, temp file, NEXT_DIST_DIR send
Status akhir	PASS WITH KNOWN RISKS

Laporan ini memuat hanya temuan yang direproduksi dan dibuktikan lewat database. Setiap perbaikan punya test yang GAGAL sebelum fix dan LULUS sesudahnya. Hipotesis yang tidak terbukti ditandai apa adanya, dan keterbatasan metode pengujian disebut di bagian 11.

EXECUTIVE SUMMARY

Audit ini tidak berangkat dari daftar bug yang sudah diketahui. Ia berangkat dari membaca kode lalu bertanya "apa yang bisa salah", menyusun hipotesis kegagalan, dan menjalankan masing-masing sampai database yang menjawab. Lima bug ditemukan, seluruhnya baru, dan seluruhnya ditutup.

Ukuran	Hasil
Kategori serangan dijalankan	8 (otorisasi, batas angka, concurrency, koreksi, replay, kebocoran, kumulatif, mutu test)
Serangan dijalankan	25 di lapisan HTTP + audit mutu test suite, seluruhnya di environment terisolasi
Bug CONFIRMED	5
Diperbaiki + regression test	5
Belum terselesaikan	0
Hipotesis tidak terbukti	1 (dicatat, lihat bagian 11)
Test BLOCKED	2 (failure injection database, lihat bagian 11)
Total test proyek	635 lulus, 0 gagal (608 → 635)

Risiko utama yang ditutup

- 1. Baris database bisa menjadi tidak terbaca selamanya.** `UPDATE saldo = saldo + ?` diterima SQLite (integer 64-bit) walau hasilnya tidak muat di kolom `Int` Prisma`. Di luar transaction, UPDATE-nya commit dan `findUnique`` atas baris itu gagal sejak saat itu. Dibuktikan langsung dengan probe, bukan disimpulkan.
- 2. Pengulangan request yang sah dijawab seolah gagal.** Penyesuaian saldo memeriksa "saldo sudah sama" sebelum jalur idempotency, sehingga pengulangan setelah response hilang dijawab 400 dengan kalimat yang menyiratkan penyesuaiannya tidak terjadi.
- 3. Pesan error yang menuntun ke jalan buntu.** Void transaksi jasa menyuruh "gunakan refund", padahal refund atas transaksi jasa mustahil dijalankan.
- 4. Test yang merah hanya antara tengah malam dan pukul 07:00.** Tiga berkas test memakai tanggal UTC sebagai hari usaha. Ditemukan bukan lewat hipotesis, melainkan karena audit ini kebetulan berjalan pukul 06:00 WIB — dan itu justru yang membuatnya layak disebut: ia sudah ada di repository dan lolos setiap kali suite dijalankan pada jam kerja.

1. SYSTEM UNDER TEST

Aplikasi kasir satu toko: Next.js 15 (App Router), TypeScript strict, Prisma + SQLite (WAL, connection_limit=1), LAN-only tanpa TLS. Seluruh rupiah integer. Tidak ada offline sync engine.

Jalur mutasi yang diaudit

```
CLIENT -> ROUTE -> requireSession/requireRole -> Zod -> SERVICE
-> idempotency fast-path -> prisma.$transaction
-> guarded UPDATE (where status lama / BETWEEN batas kolom)
-> movement/ledger row -> AuditLog -> RESPONSE
```

Endpoint diaudit	Efek
POST /api/transactions	Transaksi, stok, saldo provider, kas
POST /api/transactions/:id/refunds	Uang keluar, stok kembali, refundedQty kumulatif
POST /api/transactions/:id/void	Pembatalan, stok kembali, status pembayaran
POST /api/expenses	Kas laci (expectedCash)
POST /api/products/:id/stock-in	Stok, harga beli
POST /api/products/:id/stock-adjustment	Stok
POST /api/providers/:id/topup	Saldo provider + kas laci
POST /api/providers/:id/adjustment	Saldo provider (PIN pemilik)
POST/DELETE /api/products(/:id)	Katalog + riwayat stok
POST /api/payments/:id/confirm cancel	State machine pembayaran
POST /api/shifts/:id/close	Rekonsiliasi kas, auto-cancel PENDING

2. AUDIT METHODOLOGY

Setiap temuan melewati urutan yang sama, dan tidak satu pun dilewati: baca kode → susun hipotesis → tulis serangan → jalankan → periksa DATABASE, bukan hanya status HTTP → perbaiki → jalankan ulang → jalankan seluruh suite.

Kategori	Bentuk serangan	Berkas
Otorisasi	Tanpa session; role kasir di endpoint pemilik; PIN pemilik di body milik kasir	attack A1–A3
Batas angka	Nominal di batas Int, penjumlahan yang melampauinya, desimal, negatif, tipe salinan Basis	attack B1–B4
Concurrency	Promise.all 2 dan 5 request: penyesuaian saldo, top-up, checkout jasa, barang attack	attack C1–C4
Koreksi transaksi	Void transaksi jasa lalu menelusuri apakah jalur yang disarankan benar-benar ada	attack D1–D3
Replay & lintas pengguna	Kunci milik pengguna lain; kunci sama isi berbeda; pengulangan penyesuaian	attack E1–E3
Kebocoran data	Stack trace, path berkas, SQL, hash PIN di response error	attack F1–F2
Batas kumulatif	Tiga refund serentak melebihi qty; dua void serentak; dua penutupan shift serentak	attack G1–G5
Static sweep	Seluruh penulis status pembayaran; seluruh raw SQL; escaping literal SQLite	—

Aturan keselamatan yang dipatuhi: server presentasi di port 3000 tidak pernah dimatikan, tidak direstart, dan databasenya tidak pernah menjadi sasaran serangan. Setiap serangan menjalankan server Next.js-nya sendiri di port bebas dengan database temp, NEXT_DIST_DIR terpisah (.next-attack), dan BACKUP_DIR terpisah.

3. BUG FINDINGS

BH-01 — Nilai di luar batas kolom Int ditulis diam-diam, lalu barisnya tidak bisa dibaca lagi

Kategori	Database integrity / error handling
Severity	HIGH
Komponen	src/lib/db/provider-balance.ts, src/lib/db/stock.ts
Endpoint	POST /api/providers/:id/topup, /api/transactions, /api/products/:id/stock-in
Reproduksi	Provider dengan saldo beberapa juta, lalu top-up sebesar 2.147.483.647 (lolos batas Zod per-field). attack B3.
Diharapkan	400 dengan kalimat yang menyebut batas dan menyuruh memeriksa jumlah nol.
Aktual	HTTP 500 {"code":"INTERNAL","message":"Terjadi kesalahan di server."}. Probe terpisah membuktikan sebabnya: UPDATE
Root cause	SQLite menyimpan integer 64-bit; kolom Prisma `Int` 32-bit. Penjagaan batas hanya ada pada input per-field, bukan pada ha
Evidence	Probe: "GAGAL di queryRaw" lalu "findUnique GAGAL" pada baris yang sama — barisnya tertulis, pembacaannya tidak mun
Perbaikan	Syarat batas dipindah ke dalam SQL: AND saldo + ? BETWEEN -2147483647 AND 2147483647. Nol baris dibedakan menj
Regression test	tests/attack.test.ts B1, B1b, B3
Verifikasi	GAGAL sebelum fix (500), LULUS sesudahnya (400 + saldo tidak berubah)
Sisa risiko	Batas Int 32-bit tetap ada; yang berubah adalah cara menolaknya.

BH-02 — Pengulangan penyesuaian saldo dijawab 400 yang menyiratkan gagal, padahal berhasil

Kategori	Idempotency / kontrak API
Severity	MEDIUM
Komponen	src/lib/provider/service.ts — adjustProviderBalance
Endpoint	POST /api/providers/:id/adjustment
Reproduksi	Kirim penyesuaian saldo dua kali dengan kunci dan isi yang sama. attack E3.
Diharapkan	200 dengan replayed:true dan hasil yang pertama.
Aktual	400 "Saldo tercatat sudah sama dengan saldo yang dimasukkan".
Root cause	Pemeriksaan bisnis "selisih === 0" berjalan SEBELUM jalur cepat idempotency. Request pertama yang berhasil-lah yang m
Evidence	expected 400 to be 200 — saldo di database sudah benar, jawabannya yang salah.
Perbaikan	Sidik jari dihitung tanpa menyentuh database, jalur cepat kunci dijalankan lebih dulu, baru pemeriksaan bisnis.
Regression test	tests/attack.test.ts E3
Verifikasi	GAGAL sebelum fix (400), LULUS sesudahnya (200 + replayed:true)
Sisa risiko	Tidak ada. Saldo tidak pernah berubah dua kali, baik sebelum maupun sesudah.

BH-03 — Selisih penyesuaian saldo dihitung dari pembacaan di luar transaction

Kategori	Concurrency / race condition
Severity	MEDIUM
Komponen	src/lib/provider/service.ts — adjustProviderBalance
Endpoint	POST /api/providers/:id/adjustment
Reproduksi	Lima penyesuaian serentak ke angka yang sama (attack C1). Pada environment uji, connection_limit=1 membuat pembacaan
Diharapkan	Saldo akhir = angka yang diketik pemilik.
Aktual	C1 LULUS sebelum maupun sesudah fix. Cacatnya terbukti lewat pembacaan kode, bukan lewat reproduksi: saldo dibaca d
Root cause	Baca-lalu-tulis melintasi batas transaction.
Evidence	Kode: prisma.serviceProvider.findUnique() di luar, lalu applyProviderMovement({ amountChange: selisih }) di dalam \$transac
Perbaikan	Saldo dibaca ulang DI DALAM transaction dan selisihnya dihitung di sana.
Regression test	tests/attack.test.ts C1 (penjaga terhadap regresi, bukan bukti bug)
Verifikasi	Diperbaiki bersama BH-02; C1 tetap hijau.
Sisa risiko	Ditandai jujur: perbaikan ini menutup jalur yang secara kode terbuka, tetapi tidak pernah berhasil dipicu dalam pengujian.

BH-04 — Penolakan void transaksi jasa menunjuk ke jalur yang tidak bisa ditempuh

Kategori	Functional / UX kontrak
Severity	MEDIUM
Komponen	src/lib/transaction/void-rules.ts
Endpoint	POST /api/transactions/:id/void
Reproduksi	Jual token listrik (transaksi jasa murni, tanpa baris barang), lalu coba void. attack D2.
Diharapkan	Ditolak 409 — benar — DENGAN pesan yang menyebut langkah yang benar-benar tersedia di sistem.
Aktual	Pesan berbunyi "gunakan refund". Refund dihitung dari transaction_items, dan transaksi jasa murni tidak punya satu pun: PO
Root cause	Pesan ditulis dari niat desain, bukan dari jalur yang benar-benar ada.
Evidence	Test memeriksa: kalau pesan menyebut refund, harus ada baris yang bisa di-refund. Tidak ada.
Perbaikan	Pesan menyebut langkah nyata: catat pengembalian sebagai pengeluaran kas, lalu cocokkan saldo provider lewat Saldo &ra
Regression test	tests/attack.test.ts D2 (termasuk membuktikan refund memang 400)
Verifikasi	GAGAL sebelum fix, LULUS sesudahnya
Sisa risiko	Refund untuk baris jasa memang belum ada. Batas ini diakui di docs/qris.md §4.1b, bukan disembunyikan.

BH-05 — Tiga berkas test memakai tanggal UTC sebagai hari usaha — merah tiap 00:00–07:00 WIB

Kategori	Mutu test suite / timezone
Severity	MEDIUM
Komponen	tests/e2e-jasa.test.ts, tests/e2e-qris.test.ts, tests/e2e-fase7.test.ts
Endpoint	— (cacat pada test, bukan pada aplikasi)
Reproduksi	Jalankan suite antara pukul 00:00 dan 07:00 WIB. Ditemukan tanpa sengaja: audit ini berjalan pukul 06:08 WIB, saat tanggal
Diharapkan	Hasil test tidak bergantung pada jam berapa ia dijalankan.
Aktual	e2e-jasa nomor 18 gagal dengan "expected 0 to be 16500": laporan diminta untuk tanggal UTC, sementara seluruh transaksi
Root cause	new Date().toISOString().slice(0, 10) menghasilkan tanggal UTC. Aplikasinya mengelompokkan seluruh laporan dengan bus
Evidence	node -e: UTC "2026-09-20" vs WIB "2026-09-21" pada saat audit berjalan.
Perbaikan	Helper hariUsaha() memakai toLocaleDateString("en-CA", {timeZone:"Asia/Jakarta"}) di ketiga berkas, ditambah penjaga str
Regression test	tests/test-hygiene.test.ts — "tidak ada test yang memakai tanggal UTC sebagai hari usaha"
Verifikasi	Penjaganya dibuktikan: pola lama dikembalikan sementara ke satu berkas, penjaga GAGAL; dipulihkan, penjaga LULUS.
Sisa risiko	Test yang merah hanya di jam tertentu mengajari orang mengabaikan warna merah. Penjaga ini mencegah pola yang sama

4. TEST COVERAGE MATRIX

Diisi dari hasil test yang benar-benar dijalankan, bukan dari keberadaan berkas test.

Kategori serangan	Test	Pass	Fail	Blocked
Otorisasi (tanpa session, role salah)	3	3	0	0
Batas angka & overflow	5	5	0	0
Concurrency	4	4	0	0
Koreksi transaksi jasa	3	3	0	0
Replay & lintas pengguna	3	3	0	0
Kebocoran data pada error	2	2	0	0
Batas kumulatif (refund/void/shift)	5	5	0	0
Mutu test suite (penjaga struktural)	1	1	0	0
TOTAL serangan	26	26	0	0

Matriks regresi per endpoint

Area	Fungsional	Negatif	Duplikat	Serentak	Retry	Rollback	Otorisasi
Checkout	PASS	PASS	PASS	PASS	PASS	PASS	PASS
Refund	PASS	PASS	PASS	PASS	PASS	PASS	PASS
Pengeluaran	PASS	PASS	PASS	PASS	PASS	N/A	PASS
Barang masuk	PASS	PASS	PASS	PASS	PASS	PASS	PASS
Penyesuaian stok	PASS	PASS	PASS	NOT TESTED	PASS	PASS	PASS
Jasa pembayaran	PASS	PASS	PASS	PASS	PASS	PASS	PASS
Saldo provider	PASS	PASS	PASS	PASS	PASS	PASS	PASS
QRIS	PASS	PASS	PASS	PASS	PASS	PASS	PASS
Tutup shift	PASS	PASS	N/A	PASS	N/A	PASS	PASS

"NOT TESTED" ditulis apa adanya: penyesuaian stok belum diuji di bawah request serentak. Penulis stoknya sama persis dengan barang masuk (applyStockMovement dengan UPDATE atomic), yang sudah diuji serentak di C4 — tetapi kesamaan implementasi bukan bukti, jadi ia tidak ditandai PASS.

5. TRANSACTION INTEGRITY

Invarian	Serangan	Hasil
Kunci sekali-pakai wajib di seluruh endpoint	Request tanpa kunci ke 5 endpoint	PASS — 400, nol efek samping
Pengulangan tidak menggandakan efek	Kunci sama dikirim ulang; serentak	PASS — 200 replayed, satu baris
Kunci sama isi berbeda ditolak	Nominal diubah, kunci tetap	PASS — 409
Kunci milik orang lain ditolak	Pemilik kedua memakai kunci pemilik pertama	PASS — 409, angka tidak bocor
Saldo provider tidak kehilangan update	5 checkout jasa serentak	PASS — turun tepat 5x
Stok tidak kehilangan update	5 barang masuk serentak	PASS — naik tepat 5x
Refund kumulatif ≤ qty asli	3 refund serentak @2 dari qty 3	PASS — refundedQty ≤ 3, baris konsisten
Void tidak berlaku dua kali	2 void serentak	PASS — 1 sukses, stok kembali 1x
Shift tidak bisa ditutup dua kali	2 penutupan serentak	PASS — 1 sukses
Top-up laci bukan pengeluaran	Periksa cashExpenses dan tabel expenses	PASS — expectedCash turun, expense 0
Nilai melampaui batas kolom tidak tertulis	Top-up 2.147.483.647 di atas saldo berjalan	PASS setelah fix — 400, saldo utuh

Rollback

Diverifikasi lewat efek samping, bukan lewat klaim: pada setiap penolakan di atas, jumlah baris transaksi, saldo provider, dan stok diperiksa ulang dari database dan harus identik dengan keadaan sebelum serangan. Batas transaction yang relevan adalah `prisma.$transaction` di checkout, void, refund, top-up, dan penyesuaian saldo.

6. QRIS STATE MACHINE

PENDING --> PAID | EXPIRED | CANCELLED | FAILED (via: NORMAL)

PAID ----> CANCELLED (via: VOID saja)

^ transisi RESMI, bukan penimpaan:

canTransition(from, to, "VOID") + guarded update per baris

EXPIRED / FAILED / CANCELLED: tidak ada transisi keluar, termasuk lewat void

Sejak toko memakai QRIS soundbox, checkout QRIS selesai dalam satu langkah lewat settleTransaction yang sama seperti tunai. Yang TIDAK berubah dan diuji ulang di audit ini: tidak ada yang menjadi PAID tanpa aksi manusia dengan session, pelunasan ganda ditolak, dan jalur PENDING → PAID tetap ada untuk provider dinamis nanti.

Serangan	Hasil
Dua void serentak atas satu transaksi	PASS — 1 sukses, stok kembali sekali
Konfirmasi ulang atas pembayaran PAID	PASS — 409
Pembatalan atas pembayaran PAID	PASS — 409
Konfirmasi tanpa session	PASS — 401
Dua checkout QRIS serentak, kunci sama	PASS — tepat 1 transaksi
Baris PENDING dilunaskan lewat /confirm	PASS — jalur provider dinamis utuh
Void transaksi jasa	PASS — 409 (BH-04: pesannya diperbaiki)

7. SECURITY FINDINGS

Pemeriksaan	Hasil
Endpoint provider/produk tanpa session	PASS — 401 seluruhnya
Kasir di endpoint khusus pemilik (6 endpoint)	PASS — 403 seluruhnya
PIN pemilik yang BENAR di body milik session kasir	PASS — tetap 403; PIN tidak menaikkan hak akses
Penyesuaian saldo dengan PIN salah	PASS — ditolak, saldo utuh
Kunci idempotency lintas pengguna	PASS — 409, nilai saldo orang lain tidak muncul di response
Stack trace / path berkas / SQL di response error	PASS — tidak ditemukan
Hash PIN (bcrypt \$2a/\$2b) di response	PASS — tidak ditemukan
Kolom berlebih pada GET /api/providers	PASS — hanya id, nama, jenis, saldo, urutan, aktif
Raw SQL dengan input pengguna	PASS — seluruh \$queryRawUnsafe memakai path internal; literal di-escape

Catatan yang tidak dihitung sebagai temuan: seluruh pengguna yang sudah login boleh melakukan top-up saldo provider mana pun. Itu sesuai model satu toko — tidak ada tenant, dan uang top-up diambil dari laci kasir yang bersangkutan sehingga ia justru HARUS bisa mencatatnya. Bukan IDOR.

8. ERROR HANDLING

Masukan	Jawaban	Efek samping
Nominal desimal (1000.5)	400	nol
Nominal nol dan negatif	400	nol
Biaya admin negatif	400	nol
Jenis jasa tidak dikenal	400	nol
providerId bukan UUID	400	nol
providerId tidak ada	404	nol
Tipe salah (string / array untuk angka)	400	nol
Field wajib hilang	400	nol
lines bukan array	400	nol
Total melampaui batas kolom Int	400 (setelah BH-01)	nol
Pengeluaran saat tidak ada shift terbuka	409	nol

Yang diperiksa bukan hanya status HTTP. Untuk setiap baris di atas, jumlah baris transaksi, saldo provider, dan stok dibaca ulang dari database dan harus sama dengan keadaan sebelum request.

9. REGRESSION VERIFICATION

Bug	Regression test	Sebelum fix	Sesudah fix
BH-01	attack B1, B1b, B3	FAIL (500)	PASS (400, saldo utuh)
BH-02	attack E3	FAIL (400)	PASS (200, replayed:true)
BH-03	attack C1	PASS (race tidak tereproduksi)	PASS (jalur ditutup)
BH-04	attack D2	FAIL (jalan buntu)	PASS (pesan menyebut jalur nyata)
BH-05	hygiene: tanggal UTC	FAIL (pola terdeteksi)	PASS (ketiga berkas bersih)

Seluruh suite proyek

	Sebelum audit	Sesudah audit
Berkas test	38	39
Test lulus	608	635
Test gagal	0	0
Test dihapus	—	0

10. COMMANDS & EVIDENCE

```
$ npx vitest run tests/attack.test.ts # sebelum fix
Tests 3 failed | 16 passed (19) EXIT=1
B3 -> expected 500 to be 201
D2 -> pesan menyuruh memakai refund, tetapi transaksi jasa tidak
punya baris yang bisa di-refund - jalan buntu
E3 -> expected 400 to be 200

$ npx vitest run tests/probe.test.ts # probe akar BH-01
GAGAL di queryRaw
findUnique GAGAL -> Invalid prisma.serviceProvider.findUnique()

$ npx vitest run tests/attack.test.ts # sesudah fix
Tests 25 passed (25) EXIT=0

$ npx vitest run # sebelum BH-05 diperbaiki
Test Files 2 failed | 37 passed (39)
Tests 3 failed | 630 passed (633)

$ npx vitest run # seluruh suite, sesudah
Test Files 39 passed (39)
Tests 635 passed (635)
Duration 153.22s EXIT=0

$ npx tsc --noEmit EXIT=0
$ npx next lint EXIT=0
$ NEXT_DIST_DIR=.next-build npx next build EXIT=0

$ curl http://localhost:3000/api/health 200 (server presentasi)
```

Probe pada baris 8–10 adalah berkas test sementara yang dibuat khusus untuk menemukan akar BH-01, lalu dihapus setelah penyebabnya diketahui. Keluarannya dikutip apa adanya di atas.

11. REMAINING RISKS & LIMITATIONS

Risiko / batas	Status	Catatan
Refund untuk baris jasa tidak ada	DIAKUI	Transaksi jasa murni tidak bisa dikoreksi lewat sistem. Pesan void sekarang menyebut jalur r
BH-03 tidak pernah tereproduksi	UNCONFIRMED	Cacatnya terlihat di kode dan sudah ditutup, tetapi lima request serentak tidak berhasil memi
refundedQty memakai baca-lalu-tulis	DIAKUI	Guard kumulatif bertahan pada 3 refund serentak (G2), tetapi yang menahannya sebagian a
Failure injection database	BLOCKED	Mematikan koneksi database atau menyuntikkan kegagalan di tengah transaction membutuh
Penyesuaian stok serentak	NOT TESTED	Implementasinya sama dengan barang masuk yang sudah diuji serentak, tetapi kesamaan b
Timing concurrency tidak dijamin	BATAS METODE	Promise.all mengirim bersamaan tetapi tidak menjamin interleaving tertentu. Hasil PASS ber
Perilaku hanya-produksi	TIDAK DIUJI	Database toko, WiFi toko, dan HP kasir tidak menjadi sasaran serangan. Seluruh pengujian o

12. FINAL STATUS

PASS WITH KNOWN RISKS

Status ini ditentukan dari bukti, bukan dari target. Lima bug ditemukan dan kelima ditutup dengan regression test yang gagal sebelum fix. Seluruh 635 test proyek lulus, tanpa satu pun test dihapus atau dilemahkan.

Yang membuatnya bukan "PASS" penuh ada tiga: refund untuk baris jasa memang belum ada, dua kategori pengujian ditandai BLOCKED karena membutuhkan hook yang belum ada di kode, dan satu perbaikan (BH-03) menutup jalur yang terlihat di kode tetapi tidak pernah berhasil dipicu.

Satu hal yang patut disebut terang: BH-05 ditemukan karena audit ini kebetulan berjalan pukul enam pagi. Ia sudah ada di repository dan lolos setiap kali suite dijalankan pada jam kerja. Itu ukuran yang jujur tentang seberapa jauh audit mana pun bisa menjangkau — termasuk audit ini.

Yang tidak bisa disimpulkan dari laporan ini

- Bahwa tidak ada bug lain. Yang diuji adalah hipotesis yang terpikirkan.
- Bahwa sistem aman. Yang dibuktikan adalah serangan tertentu yang ditahan.
- Bahwa perilaku di toko sama dengan di environment uji. Database toko tidak pernah menjadi sasaran serangan, dan itu memang disengaja.

Server presentasi berjalan sepanjang audit tanpa dimatikan, direstart, atau diubah konfigurasinya. Seluruh serangan dijalankan pada server dan database terpisah.